

CSS

Deep Learning Notes

Beginner → Advanced → Modern CSS

This edition includes	Purpose
Deep explanations	Understand the 'why', not only syntax
Code → Browser Output	Predict what the browser will render
Visual animation states	See before → during → after motion
Mental models	Understand Box Model, Flexbox, Grid and Cascade
Professional topics	Responsive CSS, accessibility, debugging and maintainability
Modern CSS	Custom properties, :has(), :is(), :where(), container queries

Note about animation: PDF pages are static, so real CSS animations cannot run inside the document. For animation examples, this edition uses visual state descriptions and frame-style output so you can understand exactly what the browser would show.

1. CSS Fundamentals

CSS is a declarative language used to control presentation: typography, colors, spacing, layout, responsive behavior and visual effects.

Mental model: every CSS rule answers three questions: Which elements? Which property? Which value?

```
p {  
  color: blue;  
  font-size: 20px;  
}
```

Browser Output — Basic paragraph

Text: blue paragraph

Size: approximately 20 CSS pixels

Selector: p

Declarations: color + font-size

How it works: The browser matches every p element and applies these declarations unless a stronger applicable rule wins the cascade.

CSS rule anatomy: `p { color: blue; }` contains a selector and a declaration block. Each declaration has property:value syntax and normally ends with a semicolon.

Comments:

```
/* This comment is ignored by the browser */  
.card {  
  padding: 1rem;  
}
```

Use comments to document intent, constraints or non-obvious decisions—not to explain every obvious line.

2. Ways to Apply CSS

Method	Example	Best use
Inline	<code>style="color:red"</code>	Small experiments; avoid as a project architecture
Internal	<code><style>...</style></code>	Small standalone pages
External	<code><link rel="stylesheet" href="style.css"></code>	Most maintainable project approach

External CSS: HTML loads the stylesheet and the browser applies matching rules to the document.

```
<!-- index.html -->  
<link rel="stylesheet" href="style.css">  
  
/* style.css */  
.card {  
  padding: 20px;  
  border: 1px solid #ddd;  
}
```

Browser Output — External stylesheet

CSS is separated from HTML.

The card receives padding and a border.

The same stylesheet can be reused by multiple pages.

3. Selectors — Basic to Advanced

Selectors are patterns used to identify elements.

Selector	Meaning
*	All elements
p	All p elements
.card	Elements with class=card
#main	Element with id=main
.btn.primary	Same element has both classes
.card p	Any p inside .card
.card > p	Direct-child p only
h2 + p	First p immediately after h2
h2 ~ p	Later p siblings after h2
input[type="email"]	Attribute match

```
.card p { color: blue; }  
.card > p { font-weight: 700; }
```

Browser Output — Descendant vs direct child

Any paragraph anywhere inside .card becomes blue.

Only paragraphs that are direct children of .card become bold.

How it works: A space means descendant. The > combinator means direct child. This distinction becomes important in component-heavy interfaces.

Advanced selector family: :not(), :is(), :where(), :has(), :nth-child(), :nth-of-type(), attribute selectors and combinators let you express relationships without adding unnecessary classes.

4. Colors, Backgrounds and Gradients

Common color syntaxes include named colors, HEX, RGB and HSL. Modern CSS also supports additional color functions and spaces in current browsers.

```
.box {  
  color: #fff;  
  background: rgb(37 99 235);  
  border-color: hsl(221 83% 53%);  
}
```

Browser Output — Color card

Text: white

Surface: blue

Border: blue derived through HSL

Alpha:

```
background: rgb(15 23 42 / 80%);
```

The slash syntax adds transparency. This is useful for overlays and translucent surfaces.

```
.hero {
  min-height: 260px;
  background:
    linear-gradient(135deg, #0f172a, #334155),
    url("hero.jpg") center / cover no-repeat;
}
```

Browser Output — Hero section

Layer 1: diagonal gradient

Layer 2: image

cover: image fills the box and may crop

center: image focal point

How it works: Multiple backgrounds are layered. The first listed background is painted above later layers.

5. Typography and Text

Typography controls how information is read. Important properties include font-family, font-size, font-weight, line-height, letter-spacing, text-align, text-decoration, text-transform, white-space and overflow-wrap.

```
.article {
  max-width: 65ch;
  font-size: 1rem;
  line-height: 1.7;
}

.article h1 {
  font-size: clamp(2rem, 5vw, 4rem);
  line-height: 1.05;
}
```

Browser Output — Readable article

Heading: fluid size with a minimum and maximum

Body: constrained line length

Line-height: 1.7 gives readable vertical spacing

Font stack:

```
font-family: Inter, system-ui, -apple-system, "Segoe UI", sans-serif;
```

The browser tries the fonts from left to right and uses an available fallback. Unitless line-height is often easier to maintain because it scales with font size.

6. Units and Values

Unit	Relative to	Typical use
px	CSS pixel	Borders and precise details
%	Reference/containing value	Flexible dimensions
em	Element font-size	Local scaling
rem	Root font-size	Typography and spacing
vw/vh	Viewport	Viewport-relative sizing
fr	Available Grid space	Grid tracks
ch	Glyph measure	Readable text width

```
.title {
  font-size: clamp(2rem, 6vw, 5rem);
}
```

Browser Output — Fluid title

Small screen → stays at least 2rem.

Middle widths → preferred 6vw influences size.

Large screen → stops at 5rem.

How it works: clamp(min, preferred, max) bounds a fluid value.

7. Box Model

The box model is the foundation of CSS sizing.

```
.box {
  width: 200px;
  padding: 20px;
  border: 5px solid;
  margin: 30px;
}
```

Browser Output — Content-box calculation

Content width = 200px

Padding = 20px on each side → +40px

Border = 5px on each side → +10px

Outer width before margin = 250px

Margin creates space outside the border

How it works: With the default content-box model, declared width measures content only.

Recommended global rule:

```
*, ::before, ::after {
  box-sizing: border-box;
}
```

With border-box, declared width/height includes content + padding + border. This usually makes responsive component sizing easier.

8. Width, Height and Overflow

```
.container {
  width: min(100% - 2rem, 1100px);
  margin-inline: auto;
}
```

Browser Output — Responsive container

Small screens → nearly full width with 1rem side space.

Large screens → stops growing at 1100px.

Auto inline margins center the container.

Use width/min-width/max-width and height/min-height/max-height to control constraints. Prefer flexible constraints over fixed heights for text-heavy content.

```
.panel {  
  max-height: 300px;  
  overflow: auto;  
}
```

overflow: auto gives scrolling only when needed. Be aware that overflow can also affect sticky positioning, clipping and containing behavior.

9. Display and Formatting Contexts

Value	Concept
block	Starts a new block-level formatting flow
inline	Participates in inline text flow
inline-block	Inline participation with a box
flex	Creates a Flex formatting context
grid	Creates a Grid formatting context
none	Removed from layout/rendering
contents	Element's own box is effectively omitted; children participate in parent context

```
.row {  
  display: flex;  
  gap: 12px;  
}  
  
.grid {  
  display: grid;  
  grid-template-columns: 1fr 1fr;  
}
```

Browser Output — Flex + Grid

Flex: one-dimensional arrangement.

Grid: two-dimensional row/column arrangement.

10. Positioning and z-index

Position	Meaning
static	Normal flow; default
relative	Remains in flow; can be offset and establish useful containing context
absolute	Removed from normal flow; positioned against a containing block
fixed	Typically attached to viewport; context can vary
sticky	Behaves normally until a scroll threshold, then sticks within its scroll container

```
.card {
  position: relative;
}

.badge {
  position: absolute;
  top: 12px;
  right: 12px;
}
```

Browser Output — Card badge

Card: remains in normal layout.

Badge: positioned at the card's top-right.

The relative card commonly provides the containing block.

z-index: controls stacking within applicable stacking contexts. A child with z-index:9999 cannot necessarily escape a parent stacking context and appear above everything.

11. Flexbox — Deep Understanding

Flexbox is one-dimensional. First identify the main axis (set by flex-direction), then the cross axis.

```
.nav {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 1rem;
}
```

Browser Output — Navigation

Main-axis distribution: space-between

Cross-axis alignment: center

Spacing: gap

Items form one flexible row unless wrapping/direction changes.

Property	Question it answers
flex-direction	Which direction is the main axis?
justify-content	How should free space be distributed on the main axis?
align-items	How should items align on the cross axis?
flex-wrap	Should items create additional lines?
gap	How much space should exist between items?
flex	How should an individual item grow/shrink from its basis?

```
.center {
  min-height: 200px;
  display: flex;
  justify-content: center;
  align-items: center;
}
```

Browser Output — Perfect centering pattern

Content is centered horizontally.

Content is centered vertically.

No manual margin calculation is required.

12. CSS Grid — Deep Understanding

Grid is two-dimensional. It lets you reason in rows and columns simultaneously.

```
.cards {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 16px;
}
```

Browser Output — Three-column grid

[Card 1] [Card 2] [Card 3]

[Card 4] [Card 5] [Card 6]

Each 1fr track receives an equal fraction of available space.

```
.cards {
  display: grid;
  grid-template-columns:
    repeat(auto-fit, minmax(220px, 1fr));
  gap: 1rem;
}
```

Browser Output — Responsive auto-fit grid

Wide → more columns fit.

Medium → fewer columns.

Narrow → cards stack as space becomes limited.

How it works: auto-fit + minmax() is a high-value responsive Grid pattern because it responds to available space.

Grid areas:

```
grid-template-areas:
  "header header"
  "sidebar main"
  "footer footer";
```

Named areas make page-level layouts readable and make relationships explicit.

13. Pseudo-classes and Pseudo-elements

```
.button:hover {
  transform: translateY(-2px);
}

.button:focus-visible {
  outline: 3px solid currentColor;
  outline-offset: 3px;
}
```

Browser Output — Interactive button

Normal: button at rest.

Hover: button lifts visually.

Keyboard focus: strong visible outline appears.

Pseudo-class examples: `:hover`, `:focus`, `:focus-visible`, `:active`, `:checked`, `:disabled`, `:required`, `:valid`, `:invalid`, `:first-child`, `:last-child`, `:nth-child()`.

Pseudo-element examples: `::before`, `::after`, `::first-letter`, `::first-line`, `::marker`, `::selection`.

```
.card::before {
  content: "";
  display: block;
  height: 4px;
}
```

Generated pseudo-elements are useful for decorative accents and simple visual effects.

14. Combinators and Attribute Selectors

```
article p {}           /* descendant */
article > p {}          /* direct child */
h2 + p {}              /* adjacent sibling */
h2 ~ p {}              /* general sibling */
```

```
input[type="email"] {}
a[href^="https"] {}
a[href$=".pdf"] {}
a[href*="docs"] {}
```

Operator	Meaning
=	Exact value
^=	Starts with
\$=	Ends with
*=	Contains substring
~=	Contains whitespace-separated word
=	Exact value or value followed by hyphen

15. Cascade, Specificity and Inheritance

The cascade resolves competing declarations. In simplified author-CSS reasoning, consider importance, cascade layers, specificity, scope/source order and inheritance rather than assuming the last rule always wins.

```
p { color: black; }
.article p { color: blue; }
#post p { color: red; }
```

Browser Output — Specificity example

A normal p → black.

A p inside .article → blue when the class rule is the strongest match.

A p inside #post → red because the ID-containing selector has stronger specificity.

How it works: When two declarations tie in the relevant cascade dimensions, later source order can decide the winner.

Selector	Illustrative specificity
p	0-0-0-1
.card	0-0-1-0
#post	0-1-0-0
#post .card p	0-1-1-1
:where(.card)	0-0-0-0

Inheritance: properties such as color and font-family commonly inherit; margin, padding and border generally do not. Explicit keywords include inherit, initial, unset and revert.

!important: use sparingly. Repeated !important usually signals a specificity/cascade architecture problem.

16. Borders, Radius, Shadows and Effects

```
.card {
  border: 1px solid #d1d5db;
  border-radius: 18px;
  box-shadow: 0 16px 40px rgb(0 0 0 / 12%);
}
```

Browser Output — Modern card

Rounded corners define the component silhouette.

A subtle border gives edge definition.

A soft shadow creates visual separation.

Other effect properties include outline, outline-offset, text-shadow, opacity, filter, clip-path, mix-blend-mode and background-blend-mode. Use effects deliberately; visual noise can reduce hierarchy.

17. Transitions and Transforms

```
.button {
  transition:
    transform 180ms ease,
    opacity 180ms ease;
}

.button:hover {
  transform: translateY(-3px);
  opacity: .92;
}
```

Browser Output — Animated hover state

Frame 0: translateY(0), opacity 1.00

Frame 1: browser interpolates values.

Frame 2: translateY(-3px), opacity 0.92

Result: a smooth lift rather than a sudden jump.

How it works: Transitions interpolate between two states. They do not create the state themselves; :hover supplies the changed values.

Transform functions: translate, scale, rotate and skew. transform-origin changes the reference point for transformations.

18. Keyframe Animations

```
@keyframes pulse {
  0% { transform: scale(1); }
  50% { transform: scale(1.06); }
  100% { transform: scale(1); }
}

.logo {
  animation: pulse 2s ease-in-out infinite;
}
```

Browser Output — Animation timeline

0%: normal size

50%: 6% larger

100%: normal size again

Repeat: infinite cycles

How it works: The browser interpolates between keyframes. The PDF shows the important states as a static storyboard.

```
@media (prefers-reduced-motion: reduce) {
  .animated {
    animation: none;
    transition: none;
  }
}
```

Respect reduced-motion preferences when motion is decorative or potentially distracting.

19. CSS Variables and Design Tokens

```

:root {
  --brand: #2563eb;
  --surface: #fff;
  --text: #111827;
  --radius: 14px;
}

.card {
  color: var(--text);
  background: var(--surface);
  border-radius: var(--radius);
}

```

Browser Output — Token-based card

The component reads reusable design tokens.

Changing `--radius` updates every component using it.

Variables follow normal cascade/scoping rules.

Fallback:

```
color: var(--brand, #2563eb);
```

If the custom property is unavailable at the point of use, the fallback value is used.

20. CSS Functions

Function	Purpose
<code>calc()</code>	Perform arithmetic with compatible CSS values
<code>min()</code>	Choose the smaller computed result
<code>max()</code>	Choose the larger computed result
<code>clamp()</code>	Bound a fluid value between min and max
<code>var()</code>	Read a custom property
<code>rgb()/hsl()</code>	Specify colors

```

.container {
  width: min(100% - 2rem, 1100px);
  margin-inline: auto;
}

```

Browser Output — Fluid centered container

Small viewport → width is viewport minus 2rem.

Large viewport → width stops at 1100px.

Auto inline margins center the container.

21. Responsive Design

Responsive design means adapting to available space and user capabilities. Avoid designing a separate stylesheet for every named device.

```
.layout {
  display: grid;
  grid-template-columns: 1fr;
}

@media (min-width: 768px) {
  .layout {
    grid-template-columns: 240px 1fr;
  }
}
```

Browser Output — Mobile-first layout

Small viewport → one column.

768px+ → sidebar + main content.

The base layout is usable before enhancements are added.

How it works: Choose breakpoints based on where your content needs a change, not because a device list says so.

Useful media features: width, height, orientation, hover, pointer, prefers-color-scheme and prefers-reduced-motion.

Images: max-width:100% prevents many overflow cases; object-fit and aspect-ratio help control media boxes.

22. Container Queries

```
.wrapper {
  container-type: inline-size;
}

@container (min-width: 520px) {
  .card {
    display: grid;
    grid-template-columns: 160px 1fr;
  }
}
```

Browser Output — Component-aware responsiveness

Narrow parent → compact card.

Wide parent → image/content two-column card.

The same component can behave differently in different containers.

How it works: Container queries are useful when a component should respond to its own available width instead of the viewport.

23. Modern Selectors and Logical Properties

```
.card:has(img) {
  padding-top: 0;
}

:is(h1, h2, h3) {
  line-height: 1.1;
}

:where(ul, ol) {
  padding-inline-start: 1.25rem;
}
```

:has() is a relational selector. :is() simplifies selector lists. :where() also simplifies selectors but contributes zero specificity, making it useful for low-specificity defaults.

```
.box {  
  margin-inline: auto;  
  padding-block: 1rem;  
  padding-inline: 1.5rem;  
  border-block-end: 1px solid;  
}
```

Logical properties describe inline/block relationships instead of hard-coding left/right/top/bottom. They are particularly useful for internationalization and RTL layouts.

24. Accessibility

- Keep keyboard focus visible; :focus-visible is useful for keyboard-focused controls.
- Do not communicate essential meaning through color alone.
- Maintain adequate foreground/background contrast.
- Respect prefers-reduced-motion.
- Avoid fixed heights that can clip text.
- Use scalable typography and sensible line lengths.
- Do not disable user zoom.
- Prefer semantic HTML and use CSS as presentation.

```
.link:focus-visible {  
  outline: 3px solid currentColor;  
  outline-offset: 4px;  
}
```

Browser Output — Keyboard focus

When the link receives keyboard focus, a visible outline appears.

outline-offset separates the indicator from the element.

25. Maintainable CSS Architecture

Layer	Examples	Purpose
Tokens	--brand, --space, --radius	Shared design decisions
Base	html, body, headings	Global defaults
Layout	.container, .stack, .grid	Structure
Components	.card, .button, .modal	Reusable UI
Utilities	.sr-only, .hidden	Small single-purpose helpers
Responsive	@media, @container	Adaptation

Keep specificity predictable. Prefer reusable classes over deeply nested selectors. Use custom properties for repeated design decisions. Avoid mixing component rules with unrelated page-specific hacks.

26. Debugging CSS Like a Developer

DevTools workflow: inspect element → matched rules → identify crossed-out declaration → computed styles → box model → Flex/Grid overlay → inspect ancestors for overflow/stacking contexts.

Problem	Inspect first
Color not changing	Specificity, cascade, inheritance, !important
Width unexpected	box-sizing, padding, border, min/max constraints

Problem	Inspect first
z-index ignored	Stacking contexts and positioned ancestors
Flex item too small	flex-basis, flex-shrink, min-width, available space
Content clipped	overflow and fixed heights
Margin behaving oddly	Margin collapse and formatting context

Temporary debugging:

```
/* Use briefly while debugging */
* {
  /* outline: 1px solid red; */
}
```

27. High-Value Practical Patterns

```
.container {
  width: min(100% - 2rem, 1200px);
  margin-inline: auto;
}
```

Browser Output — Site container

Responsive side space

Maximum readable content width

Centered layout

```
.stack {
  display: flex;
  flex-direction: column;
  gap: 1rem;
}
```

Browser Output — Vertical stack

Children are arranged vertically

gap provides consistent spacing

```
.avatar {
  width: 48px;
  aspect-ratio: 1;
  border-radius: 50%;
  object-fit: cover;
}
```

Browser Output — Circular avatar

Square ratio

Circular silhouette

Image fills box without distortion

```
.media {
  width: 100%;
  aspect-ratio: 16 / 9;
  object-fit: cover;
}
```

Browser Output — 16:9 media

Stable aspect ratio

Responsive width

Cover cropping instead of distortion

28. Mini Project — Responsive Portfolio Card

This component combines custom properties, Grid, responsive media queries, image sizing, hover states and visual hierarchy.

```
:root {
  --brand: #2563eb;
  --surface: #fff;
  --text: #111827;
  --radius: 18px;
}

.profile {
  width: min(100%, 900px);
  margin-inline: auto;
  display: grid;
  grid-template-columns: 140px 1fr;
  gap: 1.5rem;
  padding: 1.5rem;
  color: var(--text);
  background: var(--surface);
  border-radius: var(--radius);
}

.profile img {
  width: 140px;
  aspect-ratio: 1;
  object-fit: cover;
  border-radius: 50%;
}

@media (max-width: 600px) {
  .profile {
    grid-template-columns: 1fr;
    text-align: center;
  }
}
```

Browser Output — Portfolio card

Desktop: image beside content.

Mobile: image moves above content.

Tokens: colors and radius are centralized.

Grid: handles the component relationship.

Responsive rule: activates when content needs a single column.

29. Interview Revision

1. What is CSS and what does cascading mean?
2. Class vs ID selector?
3. Explain content-box vs border-box.
4. Margin vs padding?
5. What is margin collapse?
6. display:none vs visibility:hidden vs opacity:0?
7. Relative vs absolute vs fixed vs sticky?
8. What is a stacking context?
9. Flexbox vs Grid?
10. justify-content vs align-items?
11. What does 1fr mean?
12. What are pseudo-classes and pseudo-elements?
13. How does specificity work conceptually?
14. What is inheritance?
15. Why avoid excessive !important?
16. What are custom properties?
17. What does clamp() solve?
18. What is mobile-first design?
19. What is a container query?
20. What does :has() do?
21. Why use logical properties?
22. How do you make animations accessible?
23. How do you debug an overridden declaration?

30. 30-Day CSS Practice Roadmap

Days	Focus	Practice
1-3	Syntax + selectors	10 selector exercises
4-6	Colors + text + units	Profile card
7-9	Box model + display	Cards + navbar
10-13	Position + z-index	Badge + modal + tooltip
14-18	Flexbox	Navbar + hero + pricing row
19-22	Grid	Dashboard + gallery
23-24	Pseudo selectors + cascade	Interactive form
25-26	Transitions + animations	Animated cards/buttons
27	Variables + functions	Themeable UI
28	Responsive + container queries	Responsive portfolio
29	Accessibility + DevTools	Keyboard/debug audit
30	Final project	Build without tutorial

Learning rule: don't count a property as learned until you can explain what it changes, what it is relative to, and what happens when you remove or modify it.

31. Master Cheat Sheet

Area	Must know
Selectors	*, type, class, ID, descendant, >, +, ~, attribute, :not(), :is(), :where(), :has()
Box	width, min/max, height, padding, border, margin, box-sizing
Layout	display, overflow, position, inset, z-index
Flex	direction, wrap, justify-content, align-items, flex, gap
Grid	template columns/rows, repeat, minmax, auto-fit, areas, placement
Text	font-family, size, weight, line-height, letter-spacing, alignment
Visual	background, border-radius, box-shadow, opacity, filter
Motion	transition, transform, animation, keyframes
Responsive	media queries, clamp, min/max, container queries
Cascade	inheritance, specificity, source order, layers, !important
Modern	variables, logical properties, :has(), :is(), :where()
Quality	accessibility, reduced motion, DevTools, performance

Final target: open a blank HTML file and build a responsive multi-section page from scratch. If you can explain every major layout decision and debug problems using DevTools, you have moved beyond memorizing CSS into actually understanding it.